

Decrease and Conquer Algorithms

ITCS 4114: Real World Algorithms

Kalpathi Subramanian

Computer Science, UNC Charlotte

Last Modified: February, 2026

Learning Outcomes

- ♣ Explain the idea behind decrease and conquer design approach to algorithms
- ♣ Name the 3 classes of decrease and conquer algorithms
- ♣ Give examples of decrease and conquer algorithms
- ♣ Solve the complexity of decrease and conquer algorithms using the Master Theorem
- ♣ Explain the selection problem and its approach to solving it.

Decrease and Conquer Design

One Dimensional Search : Binary Search on Sorted Lists

- ◇ An algorithm design technique that **relates the solution to to a given instance to that of a smaller instance**
- ◇ Can lead to solutions that are **top-down or bottom-up**
- ◇ Three variations of the design:
 - ▷ Decrease by a constant
 - ▷ Decrease by a constant factor
 - ▷ Variable size decrease

Decrease by a Constant

Examples

- ◇ Insertion Sort
- ◇ Computing a^n , $a \neq 0$, n is a non-negative integer using the following recursive definition

$$f(n) = \begin{cases} f(n-1) \cdot a, & \text{if } n > 0, \\ 1, & \text{if } n = 0 \end{cases}$$

Decrease by a Constant Factor

Binary Search

Binary Search on Sorted Lists

- ◇ Uses a divide and conquer strategy
- ◇ $\Theta(\lg N)$ complexity
- ◇ At each step, the mid point value is compared to search key and the search moves to the left or right partition.

```
boolean Binary_Search(A[0...N-1], K) {  
    // Searches for key K on a sorted array A[0..N-1]  
    // Output: index of the key K or -1 if not found  
    left = 0; right = N-1;  
    while (l <= r) {  
        m = (l+r)/2  
        if (K == A[m]) // found it  
            return m;  
        else if (K < A[m]) // search left partition  
            r = m-1;  
        else // search right partition  
            l = m+1;  
    }  
    return -1; // not found  
}
```

Given an unordered list, what is the complexity of searching for a key using binary search?

Fake Coin Problem

Given a set of n identical coins, one is fake. Given a fair balance, identify the fake coin

- ◇ Strategy similar to binary search
- ◇ n can be odd or even.
- ◇ How would you solve the problem?
- ◇ What is the complexity of the algorithm?

Russian Peasant Multiplication

Given positive integers n and m we want to compute $m \times n$

- Use the following recurrence:

$$n \cdot m = \begin{cases} \frac{n}{2} \cdot 2m, n \text{ is even} \\ \frac{n-1}{2} \cdot m + m, n \text{ is odd} \end{cases}$$

- Example: Multiply $50 \cdot 65$

n	m	
50	65	
25	130	
12	260	(+130)
6	520	
3	1040	
1	2080	(1040)
2080 + (130+1040) = 3250		

Selection Problem

Given a set of orderable elements, find the k th smallest item

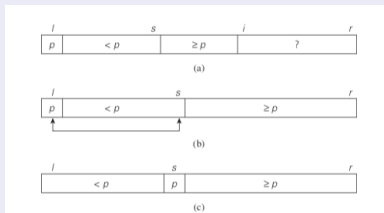
- ◇ **Application:** Find the median of a set of numbers.
- ◇ In quicksort, picking the median value as the pivot will result in $\Theta(N)$ performance, but determining the median is at least $O(N)$.
- ◇ Sorting the entire list is one way to solve the problem, **but this is overkill! We can do better!**.
- ◇ Similar to Quicksort, we will use a partitioning approach

Variable Size Decrease: Selection Problem

Algorithm to determine the s' th smallest element

- ◇ Use the Lomuto Partitioning
- ◇ Similar to quicksort, choose a pivot element and partition the elements into two subsets:

	S	
all elements are $< A[s]$	$A[s]$	all elements are $\geq A[s]$



- ◇ Once all elements are processed, partition is complete and swap the pivot with the element at the split position.
- ◇ The split position element corresponds to the **s' th smallest value in the array**

Quickselect

Algorithm

```
// Partitions subarray using first element as pivot
// Input: A subarray A[l..r]
// Output: Partition of A[l..r]
LomutoPartition(A[l..r]) {
    p = A[l]
    s = l
    for i = l + 1 to r {
        if A[i] < p)
            s = s + 1;
            swap(A[i], A[s])
    }
    swap(A[l], A[s]) // move pivot to split position
    return s
}

//Solves the selection problem
//Input: Subarray A[l..r] of array of orderable elements and
// integer k ( $1 \leq k \leq r - l + 1$ )
//Output: The value of the kth smallest element in A[l..r]
Quickselect(A[l..r], k) {
    s = LomutoPartition(A[l..r])
    if (s == (k-1))
        return A[s]
    else if (s > (l + k - 1))
        Quickselect(A[l, ..., s-1], k)
    else
        Quickselect(A[s+1, ..., r], k-1-s)
}
```

Example

Finding the median ($s = 4$)

0	1	2	3	4	5	6	7	8
s	i							
4	1	10	8	7	12	9	2	15
	s	i						
4	1	10	8	7	12	9	2	15
	s						i	
4	1	10	8	7	12	9	2	15
		s					i	
4	1	2	8	7	12	9	10	15
		s						i
4	1	2	8	7	12	9	10	15
2	1	4	8	7	12	9	10	15

$s = 2$ and smaller than 4 (median index), we go to the right partition,

0	1	2	3	4	5	6	7	8
			s	i				
			8	7	12	9	10	15
			s	i				
			8	7	12	9	10	15
			s					i
			8	7	12	9	10	15
			7	8	12	9	10	15

$s = 4$, we are done

Complexity Analysis

- Running through the Lomuto partition of an n element array takes $n - 1$ operations. In the best case, you find the solution in 1 iteration,

$$C_{best} = n - 1 \text{ operations} = \Theta(n)$$

- Can have extremely unbalanced partitions, so worst case,

$$C_{worst} = (n - 1) + (n - 2) + \dots + 1 = (n - 1)n/2 = \Theta(n^2)$$

- Average case performance is typically **linear**
- More sophisticated algorithms focus on cleverly choosing the pivot and are of linear complexity in the worst case!

Exercise

Given the sequence **9, 12, 5, 17, 20, 30, 8**, find its median using the Quickselect algorithm